CRYPTOGRAPHY

ANDREW WOOD

Contents

1. Basic Definitions 1
2. Group Theory 1
3. The Discrete Logarithm Problem 4
4. Calculating Discrete Logarithms 7
5. Primes and Primality Testing 10

1. Basic Definitions

There are two principal ways to keep a message secret:

- You can conceal the message and hope that the enemy can't find it: this is known as *steganography*.
- You can scramble the message, and hope that, if it is intercepted, the enemy will be unable to unscramble it: this is known as *cryptography*.

Definition 1.1. A *code* is a transformation of a set of characters into another set (secrecy is not necessarily important).

A *cipher*, or *cryptographic algorithm*, is a code which tries to preserve secrecy.

Cryptography is the art of devising ciphers to protect information.

Cryptanalysis is the art of breaking ciphers in order to read protected information.

Cryptology is the branch of mathematics that incorporates both cryptography and cryptanalysis.

The *plaintext* is the message to be transformed into secret form.

The *ciphertext* is the plaintext in secret form. Note that in general, Alice and Bob must assume that Eve can get her hands on the ciphertext.

Encryption is the process of applying a cipher to plaintext to produce ciphertext. This is also called *enciphering*.

Decryption is the process of applying a cipher to ciphertext to recover the original plaintext. This is also called *deciphering*.

A *key* is a secret piece of information which determines the output of a cipher.

Semantic security concerns itself with the ciphertext, and whether it reveals information about the plaintext. We can add a nonce to help. Side channel attacks refer to timing attacks. Human error is one of the biggest things we care about in security (such as social engineering attack). Mathematics is not always used in security, code words and use of language can be employed.

2. Group Theory

Definition 2.1. A *group* consists of a set of elements G together with a binary operation $\circ$ defined on G which satisfies the following axioms.

(1) The binary operation is *internal*, that is, for all $a, b \in G$, $a \circ b \in G$.

(2) The binary operation is *associative*, that is, $f \circ (g \circ h) = (f \circ g) \circ h$.
(3) There is an element $e \in G$ (the *identity*) such that $e \circ g = g \circ e = g$ for all $g \in G$.
(4) For any $g \in G$ there is some $h \in G$ (the *inverse* of g) such that $h \circ g = g \circ h = e$.

Example 2.2. The two main examples relevant to us are the following:

- $(\mathbb{Z}_m, +)$ is the group of integers modulo m with addition. When $m = p$ is a prime number this group is also denoted $(\mathrm{GF}(p), +)$, $(\mathbb{F}_p, +)$.
- $\mathbb{Z}_m^\times$ denotes the group of invertible integers modulo m under multiplication. In other words, the elements of $\mathbb{Z}_m^\times$ are
$$\{a \in \mathbb{Z}_m \mid \gcd(a, m) = 1\}.$$
When $m = p$ is prime this group is often denoted $\mathbb{F}_p^\times$, or $\mathbb{F}_p^*$.

Definition 2.3. Let G be a finite group. The *order* of G, written $|G|$, is the number of elements in G. If $a \in G$, then the *order* of a, written $|a|$, is the smallest positive integer n such that $a^n = e$, where e is the identity.

Note. For finite groups, such an exponent will always exist.

Definition 2.4. Let G be a group. We consider H to be a *subgroup* of G, written $H \leq G$, if $H \subseteq G$ and H satisfies the axioms of a group.

Definition 2.5. Let H be a subgroup of a group G. Then for $g \in G$, a *left coset* of H with respect to g is defined
$$gH := \{gh \in G \mid h \in H\}.$$
Similarly, the *right coset* of H with respect to g is defined
$$Hg := \{hg \in G \mid h \in H\}.$$

Lemma 2.6. *An element x belongs to a coset gH if and only if $g^{-1}x \in H$, which happens if and only if $gH = xH$. Thus, distinct cosets are disjoint.*

Proof. Let gH be a left coset of a group G. Then if $x \in gH$, this is equivalent to $x = gh$ for some $h \in H$ and
$$\begin{aligned} & x = gh \\ \iff \quad & h = g^{-1}x \\ \iff \quad & g^{-1}x \in H, \\ \iff \quad & x^{-1}g \in H \\ \iff \quad & g \in xH. \end{aligned}$$

If $x = gh$, then
$$xH = (gh)H = g(hH) = gH,$$
where the last equality follows from the fact that (left) multiplication by h is a bijection from H to itself. □

Theorem 2.7. *If G is a finite group and $H \leq G$, then the order of H divides the order of G.*

Proof. Let G be a group and H be a subgroup of G. We let $g_1H, g_2H, \ldots, g_nH$ be the distinct, disjoint cosets of H. We get that $h \mapsto g_ih$ is an invertible map from

$H \to g_i H$, and so we get $|g_i H| = |H|$. Since the set of disjoint cosets is a partition of G,

$$|G| = \left|\bigcup_{i=1}^{n} g_i H\right| = \sum_{i=1}^{n} |g_i H| = \sum_{i=1}^{n} |H| = n|H|,$$

and thus not only have we show that $|H|$ divides $|G|$, but we also get that $|G| = [G : H]|H|$. □

Definition 2.8. For an element $a \in G$, let

$$\langle a \rangle = \{a^n \mid n \in \mathbb{Z}\}.$$

Then $\langle a \rangle$ is a subset of G. In fact, $\langle a \rangle$ is a subgrou; of G which we call the *cyclic subgroup of G generated by a.*

Proposition 2.9. *$\langle a \rangle$ is a subgroup of G.*

Proof. We get that $\langle a \rangle$ has the identity, inverses, and closure since we take n to be the order of a such that $a^n = e \in \langle a \rangle$, for any $a^m \in \langle a \rangle$ we get that $a^{-m} \in \langle a \rangle$ and closure follows as well. □

Lemma 2.10. *Let $a \in G$ be an element in a group and suppose $|a|$ is finite. Then $\langle a \rangle = \left\{e, a, a^2, \ldots, a^{|a|-1}\right\}$, and all of these elements are distinct.*

Proof. Let n be the order of a. If $a^i = a^j$, then $a^{i-j} = e$. Without loss of generality, assume $i \geq j$. We get that $i - j = kn + r$ for some integers k and $0 \leq r < n$ using the division algorithm. Hence, $a^{kn+r} = a^r = e$ and so $r = 0$, for otherwise we get a contradiction (r would be the order of a). Thus, $i \equiv j \mod n$ if and only if $a^i = a^j$. So, $\left\{e, a, a^2, \ldots, a^{n-1}\right\}$ are all unique and make up $\langle a \rangle$. □

Corollary 2.11. *Let G be a finite group. If $a \in G$ then $|a|$ divides $|G|$.*

Proof. We get that $\langle a \rangle \leq G$ and its order is the same as a. By Lagrange's Theorem, the desired result follows. □

Corollary 2.12. *Let G be a finite group. If $a \in G$ then $a^{|G|} = e$.*

Proof. Suppose the order of a is n. Since n divides $|G|$, $|G| = nk$ for some integer k. Hence, $a^{|G|} = a^{nk} = (a^n)^k = e^k = e$. □

Corollary 2.13. *For p prime and $a \neq 0 \mod p$, $a^{p-1} = 1 \mod p$.*

Proof. The order of $\mathbb{F}_p$ is $p - 1$. Trivially, $a^{p-1} = 1$due to the result above. □

Definition 2.14. A group G is *cyclic* if there exists $a \in G$ such that $G = \langle a \rangle$. We say such an element a is a *generator* for G.

Note. In a cyclic group the binary operation *commutative*, that is, $ab = ba$ for all $a, b \in G$.

Theorem 2.15. *Let G be a finite group and $g \in G$. Then $G = \langle g \rangle$ if and only if $|G| = |g|$.*

Theorem 2.16. *Let p be prime. Then $\mathbb{F}_p^\times$ is a cyclic group.*

Note. A generator for the cyclic group $\mathbb{F}_p^\times$ under multiplication modulo p is called a *primitive* element (also primitive root modulo p).

3. The Discrete Logarithm Problem

Discrete logarithms are the group-theoretic analogues of ordinary logarithms, defined as follows.

Definition 3.1. Let G be a finite cyclic group with generator a, and let $b \in G$. We say that $x \in \{0, 1, 2, \ldots, |G| - 1\}$ is the *discrete logarithm* (or *discrete log*) of b *to base* a if

$$a^x = b,$$

which we write $x = \log_a b$. The *Discrete Logarithm Problem*, DLP: given a and b, find $x = \log_a b$.

Note. We know that $\log_a b$ is unique since $e, a, a^2, \ldots, a^{|a|-1}$ are all distinct.

Suppose G is a cyclic group $G = \langle a \rangle$. Given an integer n we can quickly, i.e. in polynomial time, compute a^x (using a method *square and multiply*). However, given $b \in G$ there is no known efficient method that works for general G which can compute $\log_a(b)$. Thus there is cryptographic interest in the DLP. However, it is important to note that it has not been *proved* that the DLP is hard for $\mathbb{F}_p^\times$ (or for any other group).

3.1. Finding Generators.

How do we find a generator of a cyclic group if we're not explicitly given one?

As we will see for the *Diffie-Hellman Key Exchange* and other cryptosystems, we are not always given the generators of our finite cyclic group. While there is no explicit formula for finding generators, there are methods/algorithms for computing them. In this section we will study generators a little more closely, starting with the number of generators in a given group.

Theorem 3.2. *Let G be a finite cyclic group with generator a, and let $m \in \{0, 1, 2, \ldots, |G| - 1\}$. Then a^m is a generator for G if and only if* $\gcd(m, |G|) = 1$.

Proof. Since a is a generator, its order is the same as G. Hence, $a^i = a^j$ if and only if $i \equiv j \mod |G|$.

($\Longrightarrow$) Suppose $\langle a^m \rangle = G$. Then there exists $n \in \mathbb{Z}$ such that $(a^m)^n = a$. The claim implies $mn \equiv 1 \mod |G|$, a multiplicative inverse. This implies $\gcd(m, |G|) = 1$.

($\Longleftarrow$) Suppose $\gcd(m, |G|) = 1$. This implies that there exists $n \in \mathbb{Z}$ such that $mn \equiv 1 \mod |G|$, that is, exists multiplicative inverse. Any element of G is the form of a^k for some integer k and so $a^k = a^{kmn} = (a^m)^{kn} \in \langle a^m \rangle$. Thus, $G \subseteq \langle a^m \rangle$ and we know that $\langle a^m \rangle \subseteq G$ such that $G = \langle a^m \rangle$. □

Definition 3.3. Let $n \in \mathbb{Z}^+$. *Euler's phi function*, $\phi(n)$ or $\varphi(n)$, is the number of positive integers a with $1 \leq a < n$ such that $\gcd(a, n) = 1$.

Theorem 3.4. *We get*

$$\varphi(p_1^{r_1} \ldots p_n^{r_n}) = p_1^{r_1 - 1}(p_1 - 1) \ldots p_n^{r_n - 1}(p_n - 1).$$

Proof. Base Case: $n = p^r$ (where p is prime). There are p^{r-1} multiples of p (up to p^r). So $\varphi(p^r) = p^{r-1}(p - 1)$.

The CRT gives m_1, m_2 relatively prime where

$$\mathbb{Z}_{m_1 m_2} \to \mathbb{Z}_{m_1} \times \mathbb{Z}_{m_2}$$

is a bijection. That is,

$$a \mapsto (a \mod m_1, a \mod m_2)$$

where a is a unit (ie elements which have mult. inverse). Then

$$\mathbb{Z}_{m_1 m_2}^\times \to \mathbb{Z}_{m_1}^\times \times \mathbb{Z}_{m_2}^\times$$

is still a bijection. The order of LHS is $\varphi(m_1 m_2)$ and RHS is $\varphi(m_1) \cdot \varphi(m_2)$ which implies (since it is a bijection) that they are equal thus proving our result by mathematical induction. □

Note. We need $\gcd(m_1, m_2) = 1$. Despite this result, we will use:

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

which can be proven to be equal rather trivially.

Corollary 3.5. *There are exactly $\phi(|G|)$ generators for G (if G is cyclic).*

Proof. Every element of G is a^m for some $m \in \{0, 1, \ldots, |G| - 1\}$. Recall a^m is a generator if and only if $\gcd(m, |G|) = 1$, which is precisely what $\phi(|G|)$ counts. □

It can be shown that

$$\frac{\phi(n)}{n} \geq \frac{1}{6 \ln(\ln(n))},$$

for all integers n, where $n \geq 5$. This means that we can expect to quickly find a primitive element of G by random guessing, even if $|G|$ is large.

How do we recognise a generator when we see it? We have two options:

(1) Given G, calculate $a, a^2, a^3, \ldots$ until e. If $|G|$ is large, this takes a long time.
(2) Using Lagrange's Theorem, we know the order of an element divides the order of a group. Thus, only test $a^m = e$ for m that divides the order of a group G.

Recall that the order of $g \in G$ is $|g| = \min\{m \in \mathbb{Z}^+ \mid g^m = e\}$. Hence the most obvious way to verify that $|g| = |G|$, is to compute $g, g^2, g^3, \ldots, g^{|G|-1}$. (That is, to adopt the brute force approach). However this is impossible if $|G|$ is large.

We can immediately improve on this naive algorithm by recalling that $|g|$ divides $|G|$ by Lagrange's Theorem. Hence, we just need to check $g^{|G|/p}$ as p ranges over the prime factors of $|G|$.

More efficient: Check $a^{|G|/p} \neq e$ for each prime dividing $|G|$. If $m \mid |G|$, $m \neq |G|$, there exists p such that $m \mid \frac{|G|}{p}$ then if $a^m = e$, $a^{|G|/p} = e$ too.

We now have an algorithm for finding a generator g of a finite cyclic group G. Let G be a cyclic group. Then:

(1) Let $n = |G|$. Factorise n to obtain factors $n = p_1^{\alpha_1} \ldots p_r^{\alpha_r}$.
(2) Choose random $g \in G$.
(3) For all i where $0 \leq i \leq r$, compute g^{m_i} where $m_i = \frac{n}{p_i}$. If $g^{m_i} = 1$ then g is not a generator so: GOTO STEP 2.
(4) Since $g^{m_i} \neq 1$ for all $0 \leq i \leq r$, g is a generator: STOP.

Note. If $|G|$ is large then we might not be able to factorise $|G|$. This stops us from using the above algorithm to find a generator for G.

3.2. **Diffie-Hellman Key Exchange.** The discrete log problem forms the basis of numerous cryptographic protocols. In this section, we consider one well-known cryptosystem.

This is the original public-key idea, developed in 1976 by Whitefield Diffie and Martin Hellman (and Ralph Merkle) and provides a mechanism for secure key exchange in symmetric key cryptosystems. Even though RSA was developed soon after, Diffie-Hellman Key Exchange is still very much in use today. This is because, in reality, symmetric ciphers are generally much faster than asymmetric ones (for both hardware and software reasons) and so, are still widely used. The slower,

public key ciphers are often used merely to facilitate the initial secure key exchange required by symmetric key cryptosystems.

Diffie-Hellman Key Exchange: Suppose Alice and Bob want to exchange secret messages using a symmetric key cipher. They securely exchange keys as follows.

(1) Alice and Bob agree on a large finite cyclic group G and a generator $g \in G$. (Both G and g can be made public).
(2) Alice chooses a random integer $a \in \{0, 1, 2, \ldots, |G| - 1\}$, which she keeps secret. She computes $A = g^a$ and sends A to Bob.
(3) Likewise, Bob chooses a random integer $b \in \{0, 1, 2, \ldots, |G| - 1\}$ and sends $B = g^b$ to Alice.
(4) Alice computes $K = B^a$ and Bob computes $K = A^b$. Note that these are equal since
$$K = B^a = \left(g^b\right)^a = g^{ba} = g^{ab} = (g^a)^b = A^b = K.$$
Then K is the common secret key, and Alice and Bob have completed a Diffie-Hellman key exchange.

Note. In practice, given a finite cyclic group G, the element $g \in G$ chosen by Alice and Bob does not necessarily have to be a generator. An element with a large order, preferably (large) prime order, will suffice, and then Alice and Bob can work in the cyclic group $\langle g \rangle$, rather than G itself.

Aside: Elliptic curve Diffie-Hellman uses points on elliptic curve instead of $\mathbb{F}_p^\times$.

Diffie-Hellman key exchange is based on the premise that an attacker who finds out A and B cannot calculate K. So exactly how secure is the system?

Theorem 3.6. *If an attacker can solve the DLP for G, they can break Diffie-Hellman key exchange. (That is, they can find K given A and B).*

Proof. If the attacker can overhear $A = g^a$ and $B = g^b$ (messages) and can solve DLP for G, you can compute a, b. Hence, the attacker knows the secrets of Alice and Bob and so can compute $K = g^{ab}$ directly. □

Note. We have only proved that solving the DLP is a *sufficient* condition for breaking the security of Diffie-Hellman key exchange (it is not a *necessary* condition). It is therefore possible that there is a way to find K given $A$and B which does not involve solving the DLP for G. This would mean that breaking Diffie-Hellman key exchange is easier than solving DLP. It is not known whether this is the case. Computational Diffie-Hellman problem: break Diffie-Hellman.

Diffie-Hellman key exchange is vulnerable to a *man-in-the-middle attack*. Recall that in this attack, Eve is an active adversary who is not only intercepting messages between Alice and Bob, but is replacing them with her own messages. Hence when Alice sends a message to Bob, Eve intercepts it and sends her own message to Bob instead. Bob thinks it came from Alice (and vice versa). In this way, Eve establishes independent connections with Alice and Bob, and relays messages between them, making them believe that they are talking directly to each other, when in fact the entire conversation is controlled by Eve.

In terms of a key exchange, when Alice and Bob exchange keys, they actually perform a key exchange with Eve. That is, Alice and Eve do a key exchange where Eve is pretending to be Bob. Then Bob and Eve do a key exchange where Eve is pretending to be Alice. Alice and Bob think they have securely exchanged keys with each other. Eve can now intercept messages from Alice to Bob, decrypt them, read them and forward them - or different messages of her choice - to Bob (and vice versa). Alice and Bob will never know.

To prevent this kind of attack, Alice and Bob can use *authentication* to establish their identities. One way of doing this is to digitally sign messages, including the original key exchange.

Note. If you are using Diffie-Hellman in $\mathbb{F}_p^\times$, we want $p-1=2q$ where q is a Sophie germain prime. So, we do *not* want $p-1$ smooth.

3.3. **ElGamal Encryption Scheme.** The ElGamal encryption is a public key cryptosystem based on the discrete log problem. Alice and Bob pick a large prime p and a primitive root g for $\mathbb{F}_p^\times$.

- Alice's public key is $A = g^a \mod p$ where a is Alice's secret key.
- To encrypt a message m, Bob chooses a secret key b and computes $B = g^b \mod p$ and $c = mA^b \mod p$. Bob sends B and c to Alice.
- Alice can decrypt by computing cB^{-a} modulo p.

When carrying this out, g, p, A, B, and c can potentially be known to an eavesdropped, while a and b are private.

3.4. **Schnorr Signature.** The Schnorr Signature is a digital signature based on the discrete logarithm problem.

Let G be a group of prime order q with the generator g. Typically one uses what is called a *Schnorr group*, which is a subgroup of $\mathbb{Z}_p^\times$ (p prime). To generate such a group, let $p = qr+1$, and choose any $h \in \{2, 3, \ldots, p-1\}$ with $h^r \neq 1 \mod p$. The value $g = h^r \mod p$ is then a generator of a subgroup $\mathbb{Z}_p^\times$ of order q. Then let $H : \{0,1\}^* \to \mathbb{Z}_q$ be a cryptographic hash function. These make up the parameters of the digital signature.

Choose a private key $x \in \mathbb{F}_q^\times$. Then the public verification key is $y = g^x$.

To sign a message $M \in \{0,1\}^*$, we randomly choose $k \in \mathbb{F}_q^\times$ and let $r = g^k$. We let $e = H(r\|M)$, where $\|$ is concatenation, and let $s = k - xe$. The signature pair is then (s, e).

To verify, let $r_v = g^s y^e \in G$ and $e_v = H(r_v\|M) \in \mathbb{Z}_q$. If $e_v = e$, then the signature is verified. This is correct since:

$$r_v = g^s y^e = g^{k-xe} g^{xe} = g^k = r$$

and so

$$e_v = H(r_v\|M) = H(r\|M) = e.$$

The public elements are G, g, q, y, s, e, r and private elements are k, x.

4. Calculating Discrete Logarithms

In this section we will consider ways to try and solve the discrete logarithm problem. This is an example of a problem which is computationally difficult to solve (like integer factorisation) and so is used as a basis for a number of public-key cryptosystems. We repeat that no one has been able to *prove* that this is a hard problem. Moreover, it has been shown that solving this problem using a quantum computer would be easy.

Efficiency: In $\mathbb{F}_p^\times$, p prime, we need $c\ln(p)$ bits to represent messages. Polynomial time efficient means we need to do $c\ln(p)^A$ operations, whereas inefficiency means $c\sqrt{p}$ or (even worse) cp operations.

Definition 4.1. The most naive way to compute a discrete log in a group is by *brute force* otherwise known as *exhaustive search* which means simply trying all candidates until a successful one is found.

We get that $\mathbb{F}_p^\times$ is tricky ($p-1$ elements) and $(\mathbb{Z}_{p-1}, +)$ ($p-1$ elements) is easy.

The Algorithm:

(1) Let G be a cyclic group with generator a and let $x = \log_a b$.
(2) We compute $a, a^2, a^3, \ldots$ until $a^x = b$.

There are $|G|$ powers. On average, one must check roughly $|G|/2$ powers. This method is not computationally efficient.

4.1. **The Baby-step Giant-step Algorithm.** The baby-step giant-step algorithm is a deterministic algorithm that computes discrete logarithms in any cyclic group G.

The algorithm works as follows. Let G be a cyclic group with generator g and assume that we want to compute the discrete logarithm of $b \in G$ to base g. In other words, we want to find the integer x where $x = \log_g b$.

We let $m = \left\lceil \sqrt{|G|} \right\rceil$.

(1) Make a list $e, g, g^2, \ldots, g^{m-1}$ (can be long).
(2) See if the list from (1) overlaps with $b, bg^{-m}, bg^{-2m}, \ldots, bg^{-(m-1)m}$. To be computationally efficient, we may stop generating this list when we find an overlap.
(3) Once we find the overlap, $bg^{-km} = g^i$ for some k, i and so $b = g^{km+i}$. Thus, $\log_g b = km + i$ and we add multiple of $|G|$ so in range $0 \leq x < |G|$.

Thus, we get that $x \equiv km + i \mod |G|$. We need to store $\approx \sqrt{|G|}$ elements of G (can be rather memory intensive). We need to multiply $\approx 2\sqrt{|G|}$ times.

4.2. **Discrete Logarithms in $\mathbb{Z}_n$.** The algorithms presented above have exponential running time. They work in arbitrary cyclic groups. But not all groups are created equal. In fact, for groups with certain properties there are much more efficient algorithms. This means that you need to be very careful when selecting a group to use for purposes such as Diffie-Hellman or Elgamal.

Example 4.2. Consider discrete logarithms in the additive group $(\mathbb{Z}_n, +)$. Suppose $a \in \{0, \ldots, n-1\}$ is a generator and you want to solve the DLP $x = \log_a(b)$ for some given b. What can we do?

Since a is a generator, $\gcd(n, a) = 1$. So, we get that a is invertible in $\mathbb{F}_p^\times$. Then take $ak = 1$ for multiplicative inverse k. That is, $a^k = 1$ under our additive group. It follows that $a^{bk} = b$ and so $x = bk \mod n$.

4.3. **The Chinese Remainder Theorem.** In the next subsection we will consider another algorithm for computing discrete logarithms that exploits certain properties of the group. It is based on an important structural result in algebra commonly referred to as the Chinsese Remainder Theorem. Let us start with a simple example.

Example 4.3. We will find all integer solutions to the system of congruences

$$x = 1 \mod 5$$
$$x = 9 \mod 11$$

Then $x = 5k+1 = 11m+9$ for some m, k. Then $5k - 11m = 8$. Using the Euclidean algorithm, we find that $11 - 2 \times 5 = 1$ which then implies $k = -16$ and $m = -8$. Hence, $x = -79 = 31 \mod 55$.

Note. The first recorded instance of a problem of this type appears in a 3rd century Chinese manuscript "Sun Tzu Suan Ching" [Master Sun's Mathematical Manual], hence the name of the theorem below.

The Chinese Remainder Theorem generalizes this example. It says that a system of congruences modulo integers which are pairwise coprime always has exactly one solution modulo the product of the moduli. Moreover, the proof gives an algorithm for computing this solution using the Euclidean Algorithm.

Theorem 4.4. *Let $m_1, m_2, \ldots, m_k$ be a collection of pairwise relatively prime integers. This means that*

$$\gcd(m_i, m_j) = 1$$

for $i \neq j$. Let $a_1, \ldots, a_k$ be arbitrary integers. Then the system of simultaneous congruences

$$x = a_1 \mod m_1, x = a_2 \mod m_2, \ldots, x = a_k \mod m_k$$

has a solution $x = c$. Moreover, if $x = c'$ is another solution, then $c = c' \mod m_1 m_2 \ldots m_k$.

Proof. We define the function $f : \mathbb{Z}_{m_1 \ldots m_k} \to \mathbb{Z}_{m_1} \times \mathbb{Z}_{m_2} \times \ldots \times \mathbb{Z}_{m_k}$ as $f(a) = (a \mod m_1, a \mod m_2, \ldots, a \mod m_k)$.

If $f(a) = f(b)$ then $a \equiv b \mod m_i$ which happens if and only if $m_i \mid a - b$ for all $1 \leq i \leq k$. Since each m_i are relatively prime, $m_1 m_2 \ldots m_k \mid a - b$ which happens if and only if $a \equiv b \mod m_1 m_2 \ldots m_k$. Thus, $a = b$ and so f is injective (one-to-one).

It follows that f is bijective. This is because we have shown f to be injective, and we know that the domain and codomain have the same size (it's automatically bijective, only valid for finite sets).

So, f is surjective (onto). That is, for $(a_1, \ldots, a_k) \in \mathbb{Z}_{m_1} \times \ldots \times \mathbb{Z}_{m_k}$ there exists $x \in \mathbb{Z}_{m_1 \ldots m_k}$ such that $f(x) = (a_1, \ldots, a_k)$. That is, $x \equiv a_1 \mod m_1$, $x \equiv a_2 \mod m_2$ and so on. This solution is unique, thus we are done. □

Example 4.5. We will find all integer solutions to the congruence

$$x^2 = 4 \mod 70 = 2 \cdot 5 \cdot 7.$$

We get that $x^2 = 0 \mod 2$ and so $x \equiv 0 \mod 2$, $x^2 = 4 \mod 5$ and so $x = \pm 2 \mod 5$, and $x^2 = 4 \mod 7$ and so $x = \pm 2 \mod 7$.

For instance, we can take $-2 + 5k = x = 2 + 7m$ which gives us $5k - 7m = 4$ for some integers k, m. Using Euclidean, we get that $7 - 5 = 2$ and so we can get that $m = -2$ and $k = -2$. That is, $x = -12 = 58 \mod 70$. So, $x = \pm 2, \pm 58 \mod 70$.

Note. In the example above while each congruence has (at most) two solutions there are 4 simultaneous solutions. An integer can have more than 2 square roots modulo N when N is not prime.

4.4. **The Pohlig-Hellman Algorithm.** The Pohlig-Hellman Algorithm is an algorithm for computing discrete logarithms based on the Chinese Remainder Theorem. It applies to the discrete logarithm problem in any group G and provides a significant speedup when the order $N = |G|$ of G is *smooth*, i.e., has no large prime factors. That is, the group is a product of small primes and so for $G = \mathbb{F}_p^\times$ we want $p - 1$ to be a product of small primes.

Note. For the discrete logarithm problem in $\mathbb{F}_p$, the relevant group is $G = \mathbb{F}_p^\times$ which has order $p - 1$. Thus it is the prime factors of $p - 1$ that play a role, not those of p. Don't be fooled into thinking that the Chinese remainder theorem won't help Eve because you are working in a field of prime order!

The algorithm is as follows. Input: G a cyclic group of order N with prime factorisation

$$N = \prod_{i=1}^{k} q_i = \prod_{i=1}^{k} p_i^{e_i},$$

where $a \in G$ is a generator of G, and $b \in G$ is any element. Note that $q_i = p_i^{e_i}$ and for $i \neq j$ we get $\gcd(q_i, q_j) = 1$. We aim to output $x = \log_a(b)$ (i.e., solve the DLP).

(1) Compute $a_i = a^{N/q_i}$ and $b_i = b^{N/q_i}$ for $i = 1, \ldots, k$.

(2) Compute $x_i = \log_{a_i}(b_i)$ using some other methods for $i = 1, \ldots, k$ (e.g. baby-step big-step algorithm, index calculus algorithm, brute-force).
(3) Find $x \in \{0, \ldots, |G| - 1\}$ such that $x = x_i \mod q_i$ for $i = 1, \ldots, k$ (by Chinese Remainder Theorem; CRT). We then output x (as it is the DL).

Theorem 4.6. *The Pohlig-Hellman Algorithm computes discrete logarithms.*

Proof. Recall $a_i = a^{N/q_i}$ and $b_i = b^{N/q_i}$. We make two claims:

(1) a_i has order q_i. We know that $a_i^{q_i} = a^N = e$ by Lagrange's Theorem. If $m < q_i$, then it follows $a_i^m = a^{N/q_i \cdot m} \neq e$ because a has order N and $\frac{N}{q_i} m < N$ which then certainly implies a_i has order q_i.
(2) $b_i \in \langle a_i \rangle$. That is, b_i is a power of a_i. This is because $b = a^m$ (since a is a generator of G) which then implies $b^{N/q_i} = a^{mN/q_i}$ and so $b_i = a_i^m \in \langle a_i \rangle$.

We note that claim (2) shows "computing $\log_{a_i}(b_i)$" makes sense.

Compute in cyclic group $\langle a_i \rangle$ and suppose we get that $x_i = \log_{a_i}(b_i)$. Suppose that $y = \log_a(b)$ (which is what we want). Then $b_i = a_i^y = a_i^{x_i}$ and so we get $y = x_i \mod q_i$. Thus $y = x \mod q_1 \ldots q_k = N$ so x is the discrete logarithm. □

We make a few remarks:

(1) The algorithm assumes a factorisation of N is given. While factorisation can also be a hard problem, it should be noted that in instances where the Pohlig-Hellman algorithm is useful the factorisation will be comparatively easy to find, because there will be lots of small prime factors. So, if $|G| = q_1 q_2 \ldots q_k$ with q_i small, then the algorithm is efficient. Otherwise, if $|G|$ is pq with pq large primes then the algorithm could be very inefficient.
(2) Most of the time will be spent on computing the discrete logarithms in smaller groups in step (2). Not great if $|G|$ is large.
(3) There are many other algorithms for calculating the discrete logarithm, with more being developed all the time. Two that are quite notable are
   - Index calculus: this is a subexponential algorithm based on techniques from linear algebra.
   - Shor's quantum algorithm: this is a polynomial time algorithm, but requires a quantum computer. This is likely to make classical Diffie-Hellman and discrete logarithm problem based cryptosystems unsecure in the near future.

The Pohlig-Hellman algorithm shows that the DLP in a group G is not secure when $N = |G|$ has many small prime factors. For discrete logarithms in $\mathbb{F}_p^\times$, we have $N = p - 1$ which can have many small prime factors even when p is a very large prime.

Example 4.7. Consider the discrete logarithm problem in $G = \mathbb{F}_p^\times$ where $p = 27! + 1$. (p is a 29 digit prime number). 2 is a generator for G and it takes our Sage server 0.01s of CPU time to compute $\log_2(100)$ in G using Pohlig-Hellman. This is because the largest prime factor of $p - 1$ is 23. By way of contrast, the next prime after p is $q = p + 46$. Our Sage server is not able to compute discrete logarithms in $\mathbb{F}_q^\times$ in any reasonable amount of time (though 30-digit primes are far from secure when attacked with more powerful machines).

5. Primes and Primality Testing

In setting up cryptographic systems like RSA or Diffie-Hellman Key Exchange in $\mathbb{F}_p^\times$, we need to be able to generate large primes. However there is still a lot that we do not know about prime numbers and to date there is no known formula for producing them. Therefore, in practice, we generate large random numbers and then test them for primality. In order for this to work, we need the following:

(1) To know that prime numbers are reasonably common (so that random guessing has a good chance of success).
(2) Good algorithms for checking whether a given number is prime.

5.1. **Prime Numbers.**

Definition 5.1. A positive integer $p \geq 2$ is *prime* if its only positive integer divisors are 1 and p.

Definition 5.2. All positive integers greater than 1 that are not prime are called *composite*. Thus, if n is composite, then $n = ab$ for some $a, b \in \mathbb{Z}^+$ where $1 < a < n$ and $1 < b < n$.

Note that from these two definitions, 1 is neither prime or composite.

Theorem 5.3. (The Fundamental Theorem of Arithmetic). *Every integer $n \geq 2$ can be written as a product*

$$n = p_1 p_2 \dots p_r$$

of prime numbers. Moreover, this prime factorisation is unique up to the order in which the p_i's are written.

Note. This is rather subtle to prove and doesn't give an efficient algorithm to find factors. We now ask, given a particular number, how can we find its prime factorisation?

In general, prime factorisation is a very difficult problem and many sophisticated prime factorisation algorithms have been devised for special types of number. Even so, the algorithms are very inefficient, which means they take a very long time to run and produce an answer. It is for this reason that prime factorisation is the basis of the RSA cryptosystem and other important cryptographic schemes. But remember, we have no *proof* that prime factorisation is difficult.

5.2. **The Number of Primes.**

Recall that we are interested in generating large primes by randomly selecting a positive integer and testing for primality. It is therefore important to know how many primes there are so that we can work out the likelihood of selecting a prime.

Theorem 5.4. (Euclid, 300BC) *There are an infinite number of primes.*

Proof. Suppose that there were finitely many primes. Then we can list them as $p_1, p_2, \dots, p_n$. We let $N = \prod_{i=1}^{n} p_i$. Consider then $N+1$. We know that $N+1$ has a prime factorisation. Then suppose that arbitrary $p_i \mid N+1$ where $1 \leq i \leq n$. We know that $p_i \mid N$ and so this implies that $p_i \mid 1$, a contradiction. Thus, there are infinitely many primes. □

While this tells us that there are infinitely many primes, it gives us no indication of which integers are prime and which are composite, or the *distribution* of prime numbers throughout the set of positive integers.

The remainder of this section will be devoted to The Prime Number Theorem (a very famous result in number theory) which describes the asymptotic distribution of the prime numbers. In other words, it gives a general description of how the primes are distributed amongst the positive integers.

Definition 5.5. For $x \in \mathbb{R}^+$, let $\pi(x)$ be the number of primes less than or equal to x. We call π the *prime counting function.*

Theorem 5.6. (The Prime Number Theorem). *The limit of the quotient of the two functions $\pi(x)$ and $\frac{x}{\ln(x)}$ as x approaches infinity is 1. This is expressed by the formula*

$$\lim_{x\to\infty} \frac{\pi(x)}{x/\ln(x)} = 1.$$

We omit the proof as they are not relevant (they use complex analysis). This tells us that

(1) $\pi(n) \approx \frac{n}{\ln(n)}$
(2) If an integer between 1 and n is chosen at random, the probability that the selected integer is prime is approximately $\frac{1}{\ln(n)}$.

The Prime Number Theorem does not tell us how *fast* the expression tends to 1 and what fluctuations there may be along the way.

Example 5.7. What is the probability that a 512 bit integer chosen at random will be prime?

We look at the number of primes up to 2^{512} and the number of numbers up to 2^{512}. That is, it is approximately $\frac{2^{512}/\ln\left(2^{512}\right)}{2^{512}} \approx \frac{1}{512\ln(2)} \approx 0.0028$.

The prime number theorem is an asymptotic result and so gives a bound on $\pi(x)$ as a direct consequence of the definition of the limit: for all $\epsilon > 0$, there is an S such that for all $x > S$,

$$(1-\epsilon)\frac{x}{\ln(x)} < \pi(x) < (1+\epsilon)\frac{x}{\ln(x)}.$$

However, other more useful bounds on $\pi(x)$ are known, and we present one in the following theorem (stated without proof).

Theorem 5.8.

$$\frac{x}{\ln(x)} < \pi(x) < 1.25506\frac{x}{\ln(x)}.$$

The left inequality holds for $x \geq 17$ and the right inequality holds for $x > 1$.

Corollary 5.9. *Let $m, n \in \mathbb{Z}$ and $17 \leq m < n$. There are $\pi(n) - \pi(m)$ primes p such that $m < p \leq n$. A further lower bound:*

$$\frac{n}{\ln(n)} - 1.25506\frac{m}{\ln(m)} < \pi(n) - \pi(m).$$

There is a connection between the Riemann Zeta function and $\pi(x)$, which means that the Riemann hypothesis, if established, would yield a far better estimate of the error involved in the Prime Number Theorem than is available today and imply a more regular distribution of prime numbers.

It was shown in 1901 that, *if and only if* the Riemann hypothesis is true, $\pi(x)$ can be approximated using the function

$$Li(x) = \int_2^x \frac{t}{\ln(t)}dt$$

we get

$$|\pi(x) - Li(x)| < \frac{1}{8\pi}\sqrt{x}\ln(x),$$

for all $x \geq 2657$. This was shown to be the "best possible" bound in 1976.

In summary, the Prime Number Theorem tells us that there are a lot of primes, which means we have a good chance of finding one by random selection. The next question is; how will we know if the number we have chosen is prime?

5.3. **Primality Testing: naive algorithms.** A *primality test* is an algorithm for determining whether a given number is prime. Unlike integer factorisation, which is considered to be a computationally difficult problem, primality testing is comparatively easy (its running time is polynomial in the size of the input). This is because primality tests do not generally calculate prime factors. Instead they rely on other methods for determining primality.

Our first two tests are deterministic tests but, as we will see, although they are always accurate, they are computationally infeasible for large primes. The other two tests are probabilistic tests. In these tests we sacrifice speed for accuracy. This means that we can quickly determine if a given number is *likely* to be prime.

The simplest way to check whether a given number n is prime is to use *trial division*. This is simply the process of systematically selecting numbers smaller than n and then seeing if they divide n. If a divisor is found, then n is composite, otherwise it is prime. The algorithm is as follows. To factor n, search $2, 3, \ldots, n-1$ to see if any divide n. If no divisors besides $1, n$ then n is prime. A bonus to this algorithm is that it gives us the prime factorisation. Ways to improve this algorithm:

(1) Only need to check prime divisors.
(2) Only need to check up to $\sqrt{n}$.

Lemma 5.10. *If n is a composite integer, then n has a prime divisor p which is less than or equal to $\sqrt{n}$.*

Proof. Let d be the smallest divisor of n greater than 1. Then d is prime because otherwise we would have a smaller divisor of n. We know that $n = dk$ for some $k \in \mathbb{Z}$. As d minimal, $k \geq d$, so $n \geq d^2$ and so $d \leq \sqrt{n}$. □

The improved algorithm is then: for all primes $p \leq \sqrt{n}$, test whether $p \mid n$. However for this algorithm to work, we first need to find the primes less than or equal to $\sqrt{n}$. We can use the procedure called *Sieve of Eratosthenes*. However, for large n, this method is still very slow.

We will now look at Wilson's theorem. Wilson's theorem gives us a necessary and sufficient condition for an integer to be prime. This theorem was stated by Ibn al-Haytham (c. 1000 AD) and, in the 18th century, by John Wilson. Edward Waring announced the theorem in 1770, although he nor his student Wilson could prove it. Lagrange gave the first proof in 1771. There is evidence that Leibniz was also aware of this result a century earlier, but he never published it.

Lemma 5.11. *Let p be a prime. There are exactly two elements in $\mathbb{Z}_p^\times$ which are their own inverse: 1 and $p-1$.*

Proof. Suppose that $x^2 = 1 \mod p$. Then, since p is prime, we must get that $p \mid x+1$ or $p \mid x-1$. That is, $x \equiv 1 \mod p$ or $x \equiv p-1 \mod p$. □

Note. Given n, if you had a way to find $a \in \mathbb{Z}_n$ such that $a^2 = 1 \mod n$, could potentially check if n prime. It is composite if you find a such that $a \not\equiv \pm 1 n$. This might even let you factor n. Suppose $n = pq$. Then $x^2 = 1 \mod pq$ happens if and only if (by CRT) $x^2 \equiv 1 \mod p$, $x^2 \equiv 1 \mod q$ and so $x = \pm 1 \mod p$ and $x = \pm \mod q$.

We can also do this with arbitrary a, b, n such that $a^2 \equiv b^2 \mod n$ with $a \not\equiv b \mod n$. Just take $\gcd(a \pm b, n)$ as non-trivial factors of n.

Theorem 5.12. (Wilson's theorem) *Let $n > 1$ be an integer. Then n is prime if and only if*

$$(n-1)! \equiv -1 \mod n.$$

Proof. ($\Longrightarrow$) Suppose that $n = p$ is prime. We get that

$$(n-1)! \equiv -(n-2)! \mod n.$$

Then for each $(n-2), (n-3), \ldots, 2$ we know that they each have inverses (which are not themselves). So, since this group is commutative, we get that $(n-2)! = 1 \mod n$. Hence,

$$(n-1)! \equiv -1 \mod n.$$

($\Longleftarrow$) We claim that if n is composite, that either $n = 4$ or there exists integers a, b with $1 < a < b < n$ such that $ab \equiv 0 \mod n$. Suppose that $n \neq 4$ and that n is composite. Then $n = ab$ for some $a, b \in \mathbb{Z}^+$ where $1 < a < n$ and $1 < b < n$. Without loss of generality, assume that $a \leq b$. If $a = b$, then we know $2 < a = b$ and so we can take $1 < a < 2b < n$ such that $2ab \equiv ab \equiv 0 \mod n$. Of course, we just take $1 < a < b < n$ if $a \neq b$. Overall, this proves the claim.

Now, using our claim, it follows easily that $(n-1)! \equiv 0 \not\equiv -1 \mod n$ and so we are done. □

5.4. **Primality testing: The Fermat Test.** The Fermat primality test is a *probabilistic* test which is able to determine whether a given number is composite or a *probable prime*. This means that while it can identify numbers which are *likely* to be prime, it cannot say with absolute certainty that they are prime. The advantage of this test is that it is much faster than trial division.

The basis of this test is Fermat's Little Theorem which we give below.

Theorem 5.13. *If p is prime and a is any positive integer such that $p \nmid a$ then $a^{p-1} \equiv 1 \mod p$.*

Proof. This is merely a Corollary of Lagrange's Theorem. □

Fermat's Little Theorem can be used to quickly show that certain numbers are not prime (that is, they are composite). However, it can never be used to show that a number is prime. This is because $a^{p-1} = 1 \mod p$ is a *necessary* condition for a number being prime (if a number is prime, this will be true) but not *sufficient* (if this is true for a given number, it does not mean the given number is prime).

Hence, using Fermat's Little Theorem, it is possible to deduce whether an integer n is composite. We do so by choosing random $1 < a < n$ where $\gcd(a, n) = 1$ (if not then n is composite). If $a^{n-1} = 1 \mod n$ then n "looks" prime. Otherwise, we know n is composite.

Note. For a given a, computing $a^{p-1} \mod p$ can be performed very quickly using the so-called *square and multiply* algorithm. Simply represent $p-1$ in binary. Then square $a, a^2, a^4, \ldots$ until we can multiply to fully represent $p-1$ in binary.

Lemma 5.14. *If a is an integer for which $a^{n-1} = 1 \mod n$, then $\gcd(a, n) = 1$.*

Proof. Suppose $a^{n-1} = 1 \mod n$. Then a has a multiplicative inverse which happens if and only if $\gcd(a, n) = 1$.

Alternatively, we can prove this by observing that $a^{n-1} = 1 \mod n$ implies $ax + ny = 1$ for some integers x, y and so this gives us $\gcd(a, n) = 1$. □

We will now construct an algorithm based on Fermat's Little Theorem. However, we will require the following two definitions.

Definition 5.15. Let a be an integer where $1 < a < n-1$. If $a^{n-1} \not\equiv 1 \mod n$, a is called a *Fermat witness* for the compositeness of n.

Definition 5.16. If n is an odd composite integer and if a is an integer for which $a^{n-1} = 1 \mod n$, then n is called a *pseudoprime to the base a*.

The Fermat Test algorithm:

(1) Pick $1 < a < n$ at random. If $\gcd(a, n) \neq 1$, n is composite and STOP.
(2) Then compute $a^{n-1} \mod n$: if $\not\equiv 1 \mod n$, STOP and then n is composite (i.e. found Fermat witness).
(3) Repeat (1) and (2) k times, if witness never found, conclude that n is *probably* prime.

Problems with the Fermat test:

(1) If test says composite, n is composite. If the test says probably prime, n could be composite.
(2) How many Fermat witnesses? We claim that $H = \left\{a \in \mathbb{Z}_n^\times \;\middle|\; a^{n-1} = 1 \mod n\right\}$ is a subgroup of $G = \mathbb{Z}_n^\times$. The first case is that $H = G$. Then n is composite but we have no Fermat witnesses. The second case is that H is smaller than G. By Lagrange's Theorem, the order of H divides the order of G. So, $|H| \leq \frac{1}{2}|G|$. If $a \in H$, then a is not a Fermat witness. For k rounds of Fermat test: chance not showing n is composite is $\leq \left(\frac{1}{2}\right)^k$ (really rare).

The algorithm above will either determine that n is composite or that n is a pseudo-prime. In practice, we can specify a value k which will determine how many times we test a new base a (and this will determine the speed of our algorithm). The higher value of k, the more likely n is to be prime. However, this raises a question. If n is composite, does there exist $a \in \mathbb{N}$ where $\gcd(a, n)$ such that $a^{n-1} \not\equiv 1 \mod n$? We therefore give the following definition.

Definition 5.17. If n is pseudoprime to the base a for all integers a where $1 < a < n - 1$ and $\gcd(a, n) = 1$, then n is a *Carmichael number.*

In practice, it is very unusual for an integer to be a pseudoprime base a for one or more values of a and yet fail to be prime. This is because there are actually very few Carmichael numbers. The Carmichael numbers under 10000 are 561, 1105, 1729, 2465, 2821, 6601, 8911. In fact, as numbers become larger, Carmichael numbers become increasingly rare. For example, there are $20,138,200$ Carmichael numbers between 1 and 10^{21} (approximately one in 50 billion numbers). However, it has been shown in 1994 by Alford, Granville and Pomerance that there are infinitely many Carmichael numbers.

Theorem 5.18. *Let n be an odd, composite positive integer. If n is square-free and for every prime p dividing n, $p - 1$ divides $n - 1$, then n is a Carmichael number.*

Note. An integer n is *square-free* if there is no prime p such that $p^2 \mid n$.

Proof. Let n be an odd and composite square free integer. Suppose that for every prime p with $p \mid n$ that $p - 1 \mid n - 1$. By Lagrange's Theorem, every cyclic group will have prime order which means that the order of each element divides n. So, $|a| = p \mid n$ which implies $a^{p-1} = 1 \mod n$. Since $p - 1 \mid n - 1$, $n - 1 = k(p - 1)$ for some k which gives us that $a^{n-1} = 1^k = 1 \mod n$. Since $\langle a \rangle \leq \mathbb{F}_n^\times$, $\gcd(a, n) = 1$ and so n is a Carmichael number. □

In summary, the Fermat test can be used conclusively to show that a number n is composite. However, it can never be used to show that n is prime. For even if we run the test many times, the existence of the Carmichael numbers means that there are infinitely many composite numbers that will pass the test.

5.5. **Primality testing: The Miller-Rabin Test.** The Miller-Rabin primality test is another *probabilistic* test which is able to determine whether a given number is composite or a *probable prime.* The advantage of this test over the Fermat Test is that there is no analogue of Carmichael numbers to worry about. To understand how this test works, we need to following lemma.

Lemma 5.19. *Let p be prime. The positive integer a is its own inverse modulo p if and only if $a \equiv 1 \mod p$ or $a \equiv -1 \mod p$.*

Proof. If $x \equiv x^{-1} \mod p$ then $x^2 \equiv 1 \mod p$ and so $p \mid (x+1)(x-1)$, which implies since p is prime, $p \mid x+1$ or $p \mid x-1$. Thus, $x \equiv 1 \mod p$ or $x \equiv -1 \mod p$. □

The Miller-Rabin test uses this result to build on the idea behind the Fermat test. Specifically, once we verify the congruence $a^{n-1} = 1 \mod n$, where n is an odd integer, it is possible to consider the least positive residue of $a^{(n-1)/2} \mod n$. If we let $x = a^{(n-1)/2}$, then

$$x^2 \equiv a^{n-1} \equiv 1 \mod n.$$

If n is prime, we know that either $x = 1 \mod n$ or $x = -1 \mod n$. Hence, either $a^{(n-1)/2} = 1 \mod n$ or $a^{(n-1)/2} = -1 \mod n$. If this is not the case, then n is composite.

We formally state the Miller-Rabin test as follows.

Theorem 5.20. *Let n be an odd positive integer such that $n > 1$. We can write $n - 1 = 2^s d$ for some non-negative integers s, d where d is odd. Let a be a positive integer such that $1 < a < n - 1$. If n is prime then either*

$$a^d = 1 \mod n$$

or there exists $r \in \{0, 1, \ldots, s-1\}$ such that

$$a^{2^r d} = -1 \mod n.$$

Proof. Look at $a^d, a^{2d}, a^{4d}, \ldots, a^{2^s d} = a^{n-1} = 1 \mod n$ because n is prime.

Let j be the smallest natural number such that $a^{2^j d} = 1 \mod n$. Either $j = 0$, or $j \neq 0$. If $j = 0$, we are done. So assume $j \neq 0$. Take $r = j - 1$. Then $a^{2^r d} = 1 \mod n$ or $a^{2^r d} = -1 \mod n$. The former contradicts j being the smallest. Hence, we get the latter and so we are done. □

Definition 5.21. Let n be an odd positive integer such that $n > 1$. An integer a such that $1 < a < n - 1$ and which does not satisfy the conditions of the Theorem above is called *Miller-Rabin witness* to the compositeness of n.

Definition 5.22. If n is a composite integer and if a is an integer for which one of the conditions of the Theorem above is satisfied, then n is called a *strong pseudoprime to the base a.*

Jn order for the Miller-Rabin test to be efficient, we need to ensure there are sufficiently many Miller-Rabin witnesses for a given composite number n. The next result guarantees this is the case.

Theorem 5.23. *Let n be an odd, composite integer. Then there are at most $\frac{n-1}{4}$ integers $a \in \{1, 2, \ldots, n-1\}$ such that a is not a witness to the compositeness of n.*

That is, there are lots of Miller-Rabin witnesses. Approximately 75% or better of randomly picking one. This tells us two things:

- if n is composite then successively choosing random $a \in \{1, 2, \ldots, n-1\}$ will quickly give us a Miller-Rabin witness,
- there is no analogy of the Carmichael numbers for strong pseudoprimes.

Using these results, we have the following algorithm for testing the integer n:

(1) Pick a random $a \in \{1, 2, \ldots, n-1\}$ (without replacement).

(2) Compute $a^{s^r d} \mod n$ for all $r \in \{0, 1, \ldots, s-1\}$. If $a^d \not\equiv 1 \mod n$ and $a^d, \ldots, a^{2^{s-1}d} \not\equiv -1 \mod n$ then a is a Miller-Rabin witness and n is composite: STOP.
(3) GOTO Step (1).

If n is composite then the chance of choosing a at Step (1) such that a is not a Miller-Rabin witness is at most $\frac{1}{4}$. So after k iterations, the chance that n is composite is only at most $\left(\frac{1}{4}\right)^k$. Hence, if the algorithm runs for k iterations without stopping then we can say that n is prime with probability of at least $1 - \left(\frac{1}{4}\right)^k$. (In practice, it is even higher).

Note that the Miller-Rabin Test runs in polynomial time (in the size) of n, provided we fix the number of iterations we allow before accepting n to be prime.

A result from 2002: Agrawal, Kayal, and Saxena found a polynomial time deterministic algorithm for determining whether an integer n is prime or composite: the algorithm succeeds with probability 100%. It is very important from a theoretical point of view, but it is not used in practice as it is slower than other tests such as the Miller-Rabin Test.

Note: in rare situations, the test will provide $a^{2^j d} \not\equiv \pm 1 \mod n$ and $a^{2^{j+1}d} \equiv 1 \mod n$ and so $\gcd\left(a^{2^j d} - 1, n\right)$ lets you factor n (as non-trivial factor).